

FACULTAD DE INGENIERÍA Y CIENCIAS BÁSICAS

Ingeniería de Datos e Inteligencia Artificial

Emmanuel Escobar Santos

Curso: ETL (G51)

Documento Técnico - Entrega Final

Pipeline ETL: Vacunación & Decesos COVID-19 Ecuador

Mayo 2026

1. Introducción

Este documento describe la arquitectura, el diseño y los resultados del pipeline ETL construido para la entrega final del curso. El proyecto integra datos de vacunación COVID-19 en Ecuador con datos de decesos provenientes de un conjunto de datos público de Google BigQuery, orquesta el flujo completo con Apache Airflow, transmite métricas en tiempo real a través de Apache Kafka, y expone un dashboard reactivo con Plotly y Dash.

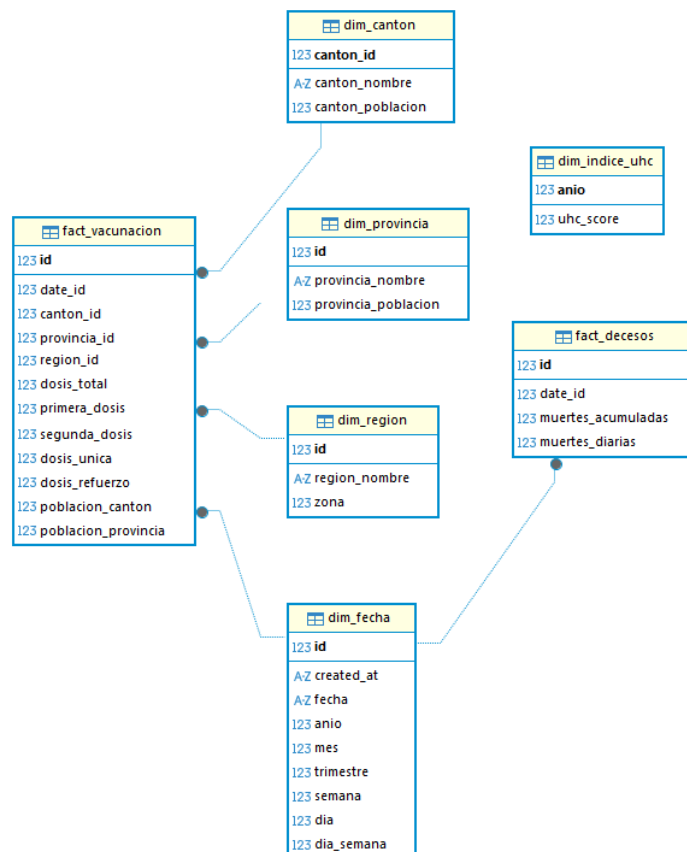
El objetivo central es demostrar la capacidad de construir un sistema de datos que sirva simultáneamente para dos propósitos: análisis histórico mediante un modelo dimensional y operación en tiempo real mediante streaming.

2. Fuentes de Datos

2.1 Fuente 1 — DWH SQLite (Base de datos propia)

El Data Warehouse desarrollado en la segunda entrega del proyecto, almacenado en Google Drive, contiene el Galaxy Schema de vacunación en Ecuador con las siguientes tablas:

- fact_vacunacion: métricas de dosis (total, primera, segunda, única, refuerzo) por cantón y fecha.
- Fact_decesos: métricas de decesos acumulados y diarios a causa del COVID-19.
- dim_fecha: calendario con granularidad diaria (año, mes, trimestre, semana, día, día de la semana).
- dim_canton: catálogo de cantones con nombre y población.
- dim_provincia: catálogo de provincias con nombre y población.
- dim_region: catálogo de regiones naturales del Ecuador con zona administrativa.
- dim_indice_uhc: índice Universal Health Coverage (UHC) de Ecuador de la OMS por año.



2.2 Fuente 2 — Google BigQuery (API pública)

Se consulta la tabla `bigquery-public-data.covid19_jhu_csse.deaths` del proyecto público de BigQuery, filtrando únicamente los registros correspondientes a Ecuador. Esta tabla contiene el acumulado diario de muertes por COVID-19 reportadas por el Johns Hopkins University Center for Systems Science and Engineering (JHU CSSE).

Los datos originales se encuentran en formato `wide` (una columna por fecha); durante la transformación se convierten a formato `long` (una fila por fecha) y se calculan las muertes diarias como diferencia entre registros consecutivos.

3. Arquitectura del Sistema

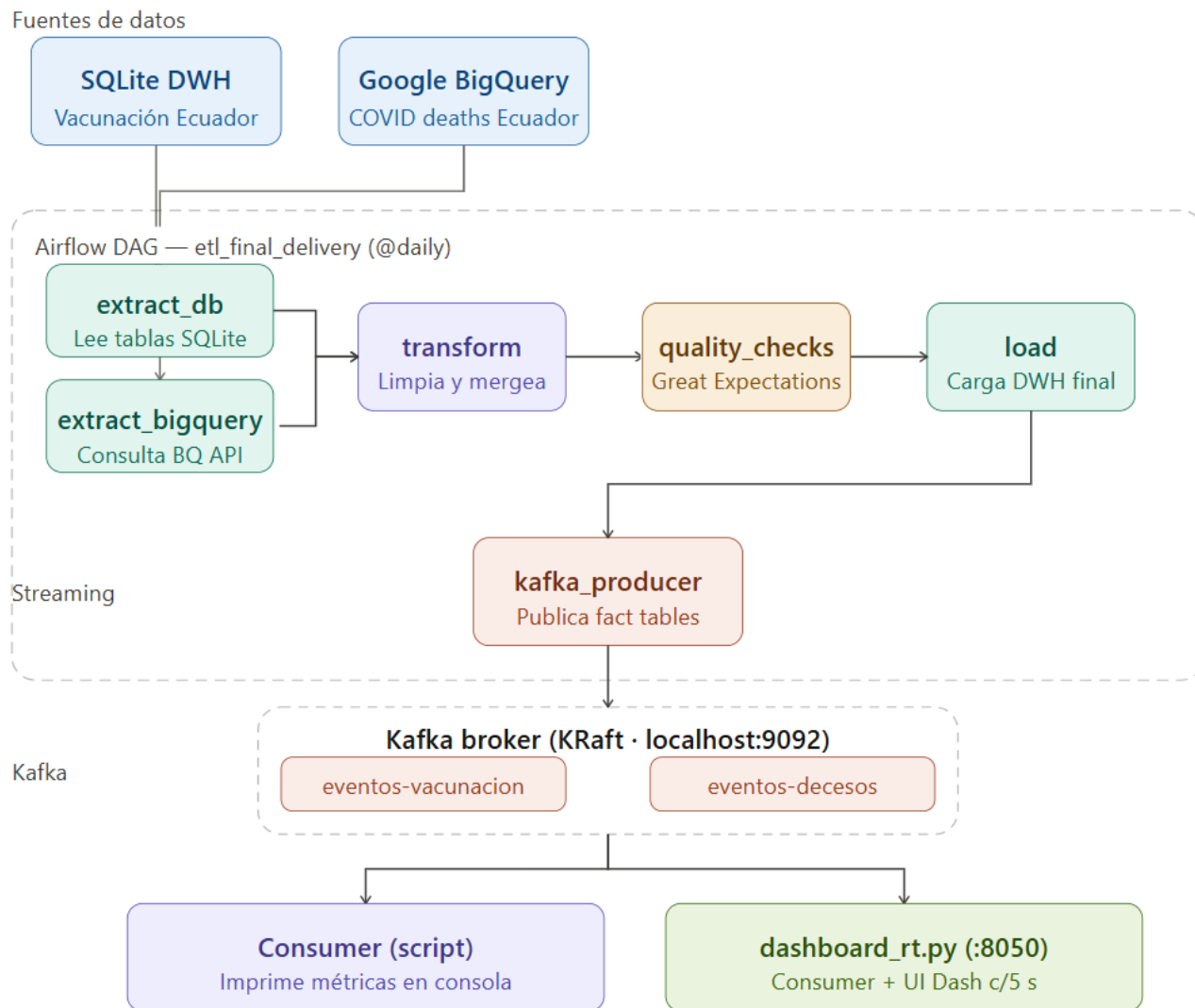
El sistema se compone de cuatro capas principales que operan de forma secuencial para el proceso batch y en paralelo para el proceso en tiempo real:

Capa	Componente	Responsabilidad
Ingesta	Airflow: <code>extract_db / extract_bigquery</code>	Extracción desde SQLite y BigQuery
Transformación	Airflow: <code>transform + quality_checks</code>	Limpieza, merge y validación con Great Expectations
Almacenamiento	SQLite DWH final (<code>dwh_final_delivery.db</code>)	Modelo dimensional (Galaxy Schema) validado y listo para análisis
Streaming	Kafka (KRaft) + Producer Airflow + Consumer Dash + Consumer Script Python	Transmisión en tiempo “real” de ambas tablas de hechos
Visualización	Plotly Dash (<code>dashboard_rt.py</code>)	Dashboard reactivo con KPIs y gráficos actualizados cada 5 segundos

3.1 Diagrama de flujo de datos

El flujo de datos sigue la secuencia:

- Las dos tareas de extracción corren primero (`extract_db` y `extract_bigquery`).
- La tarea `transform` unifica y limpia ambas fuentes, generando CSVs intermedios.
- La tarea `quality_checks` valida los CSVs con Great Expectations antes de continuar.
- La tarea `load` construye el esquema SQLite y carga todas las tablas.
- La tarea `kafka_producer` lee el DWH cargado y publica los registros en dos tópicos Kafka.
- El proceso `dashboard_rt.py` consume ambos tópicos de forma continua y actualiza la UI.



4. Diseño del DAG de Airflow

El DAG `etl_final_delivery` está configurado con `schedule_interval='@daily'`, `start_date=2018-01-01` y `catchup=False`. Tiene 3 reintentos automáticos por tarea y no depende de ejecuciones pasadas.

4.1 Descripción de tareas

Tarea (task_id)	Tipo	Descripción detallada
<code>extract_db</code>	PythonOperator	Conecta al DWH SQLite de la segunda entrega y exporta <code>fact_vacunacion</code> y las 5 dimensiones como CSV temporales en <code>/content/</code> .

extract_bigquery	PythonOperator	Usa el cliente oficial de Python para BigQuery; ejecuta una consulta SQL filtrando Ecuador y guarda el resultado crudo en /content/tmp_covid_raw.csv.
transform	PythonOperator	Limpia y tipifica cada tabla: convierte fechas, parsea enteros, elimina duplicados, valida integridad referencial por ID. Convierte datos de decesos de formato wide a long y calcula muertes diarias. Genera 7 CSVs transformados (tf_*.csv).
quality_checks	PythonOperator	Instancia un contexto de Great Expectations e itera sobre las 7 tablas aplicando expectativas: no nulos, rangos numéricos y conteo de filas. Falla si alguna expectativa no se cumple.
load	PythonOperator	Carga las dimensiones antes que las tablas de hechos para respetar la integridad referencial. Verifica rowcounts al finalizar.
kafka_producer	PythonOperator	Lee ambas tablas de hechos del DWH final y publica cada fila como mensaje JSON en el tópico correspondiente, con un delay de 1 ms entre mensajes para simular streaming continuo.

4.2 Dependencias entre tareas

```
[extract_db, extract_bigquery] >> transform >> quality_checks >> load >>
kafka_producer
```

5. Modelo Dimensional

5.1 Tablas de hechos

Tabla	Granularidad	Métricas principales	FK
fact_vacunacion	Un registro por cantón × fecha	dosis_total, primera_dosis, segunda_dosis, dosis_unica, dosis_refuerzo, poblacion_canton, poblacion_provincia	date_id, canton_id, provincia_id, region_id
fact_decesos	Un registro por fecha (Ecuador nacional)	muerteres_acumuladas, muertes_diarias	date_id (UNIQUE)

5.2 Tablas de dimensión

Dimensión	PK	Atributos clave
dim_fecha	id	fecha, anio, mes, trimestre, semana, día, día_semana
dim_canton	canton_id	canton_nombre, canton_poblacion
dim_provincia	id	provincia_nombre, provincia_poblacion
dim_region	id	region_nombre, zona
dim_indice_uhc	anio	uhc_score (índice OMS 0–100)

6. Streaming con Kafka

6.1 Configuración del broker

Se utiliza Apache Kafka en modo KRaft (sin Zookeeper), arrancado en Google Colab como proceso daemon. El broker escucha en localhost:9092.

6.2 Tópicos

Tópico	Origen	Campos relevantes
eventos-vacunacion	fact_vacunacion	id, date_id, canton_id, provincia_id, region_id, dosis_total, primera_dosis, segunda_dosis, dosis_unica, dosis_refuerzo
eventos-decesos	fact_decesos	id, date_id, muertes_acumuladas, muertes_diarias

6.3 Producer (task de AirFlow -> kafka_producer)

Itera sobre los registros extraídos de ambas tablas de hechos, serializa cada fila como JSON UTF-8 y llama a `producer.produce()` con callback de confirmación. Al finalizar, llama a `producer.flush()` para garantizar la entrega de todos los mensajes pendientes. El delay de 1 ms entre mensajes simula un flujo de datos continuo.

6.4 Consumer (integrado en `dashboard_rt.py`)

Corre en un hilo daemon con `group.id='dashboard-rt'` y `auto.offset.reset='latest'`. Almacena los mensajes entrantes en listas en memoria (vacunacion y decesos) protegidas por `threading.Lock` (evita que varios hilos accedan al mismo recurso al mismo tiempo) para garantizar acceso seguro desde el hilo de Dash.

6.5 Consumer (Script Python)

Consumer suscrito a los 2 topicos del producer que imprime en terminal los mensajes en tiempo real para confirmación visual y simulación del flujo completo.

7. Dashboard en Tiempo Real

7.1 Stack y arquitectura de la UI

El dashboard está implementado con Plotly Dash y se sirve en el puerto 8050. Un `dcc.Interval` de 5 segundos dispara el callback principal que lee los datos en memoria y regenera todos los componentes visuales.

7.2 Componentes visuales

Componente	Tipo	Datos mostrados
KPIs (5 tarjetas)	Indicadores numéricos	Eventos recibidos (vac. / dec.), dosis totales acumuladas, muertes acumuladas, tasa msg/s
g-dosis	Gráfico de barras	Dosis acumuladas por tipo (primera, segunda, única, refuerzo)
g-decesos	Gráfico de área	Muertes acumuladas en el tiempo (secuencia de mensajes recibidos)
g-provincia	Barras horizontales	Top 10 provincias por dosis totales acumuladas
g-flujo	Gráfico de líneas	Mensajes acumulados de ambos tópicos + tasa actual
tabla-ultimos	Tabla HTML	Últimos 5 registros de vacunación + últimos 5 de decesos

7.3 Lógica de acumulación

Los datos de `fact_vacunacion` son acumulados por cantón: la función `_snapshot_canton()` retiene únicamente el último registro recibido por cantón (el de mayor `dosis_total`), evitando duplicar conteos al sumar los totales. Los datos de `fact_decesos` ya contienen `muertes_acumuladas` hasta la fecha, por lo que se grafican directamente.

8. Calidad de Datos

La tarea `quality_checks` aplica el siguiente conjunto de expectativas con Great Expectations:

Tabla	Expectativas aplicadas
<code>fact_vacunacion</code>	Rowcount > 0; no nulos en id; unicidad de id; dosis ≥ 0; poblaciones ≥ 0
<code>dim_canton</code>	Rowcount > 0; no nulos e unicidad en <code>canton_id</code> ; <code>canton_nombre</code> no nulo; <code>poblacion</code> ≥ 0
<code>dim_fecha</code>	Rowcount > 0; no nulos e unicidad en id; fecha no nulo; mes 1–12; día 1–31
<code>dim_indice_uhc</code>	Rowcount > 0; no nulos e unicidad en <code>anio</code> ; <code>uhc_score</code> 0–100
<code>covid_deaths</code> (<code>fact_decesos</code>)	Rowcount > 0; fecha no nulo; <code>muertes_acumuladas</code> ≥ 0; <code>muertes_diarias</code> ≥ 0; mes 1–12

Si alguna expectativa falla, se lanza una excepción con el detalle de los errores y el pipeline se detiene antes de la carga, garantizando que el DWH final solo contenga datos válidos.

9. Visualizaciones e Insights

9.1 Dashboard estático (modelo dimensional)

Conectado al DWH SQLite final, permite analizar:

- Evolución temporal de la cobertura de vacunación.
- Distribución de tipos de dosis.

- Relación entre el índice UHC por año y el avance de la campaña de vacunación.
- Diferencias de cobertura entre regiones (Costa, Sierra, Amazonía, Galápagos).
- Cantones con mayor tasa de cobertura vacunal.
- Total de dosis por region.
- Top cantones y provincias con mayor numero de dosis aplicadas.

9.2 Dashboard en tiempo real (streaming Kafka)

El dashboard en tiempo real permite observar:

- La tasa de llegada de mensajes en msg/s, indicadora de la cadencia del producer.
- El avance acumulado de dosis conforme los registros históricos son re-emitidos por el producer.
- El avance acumulado de la distribución de las dosis.
- La cantidad de registros recibidos de vacunación y decesos.
- El comportamiento de los decesos acumulados a lo largo del tiempo.
- Las provincias con mayor volumen de vacunación, actualizándose con cada lote de mensajes recibidos.

10. Generación de Valor

Dimensión	Mecanismo técnico	Valor generado
Operacional (tiempo real)	Stream Kafka → Consumer Python → Consumer Dash, refresco cada 5 s	Permite monitorear el avance de la vacunación y la evolución de los decesos en tiempo casi-real. En un contexto productivo, un equipo de salud pública podría detectar anomalías (pico súbito de muertes, caída en la tasa de vacunación) e intervenir de inmediato siendo una gran ventaja técnica.
Analítico (batch)	Airflow DAG diario → DWH dimensional → Dashboard estático	El modelo de galaxia permite responder preguntas históricas complejas: tasas de cobertura por cantón, correlación entre índice UHC y vacunación, evolución trimestral, entre otras. El DAG garantiza que el DWH se actualiza y valida automáticamente cada día.

11. Stack Tecnológico

Capa	Herramienta
Orquestación	Apache Airflow
Entorno de ejecución	Google Colab (Python)
Almacenamiento	SQLite
Fuente cloud	Google BigQuery
Calidad de datos	Great Expectations
Broker de mensajería	Apache Kafka (KRaft)
Cliente Kafka Python	confluent-kafka
Visualización RT	Plotly Dash + Plotly
Túnel público	Cloudflare Tunnel (cloudflared)
Control de versiones	Git / GitHub